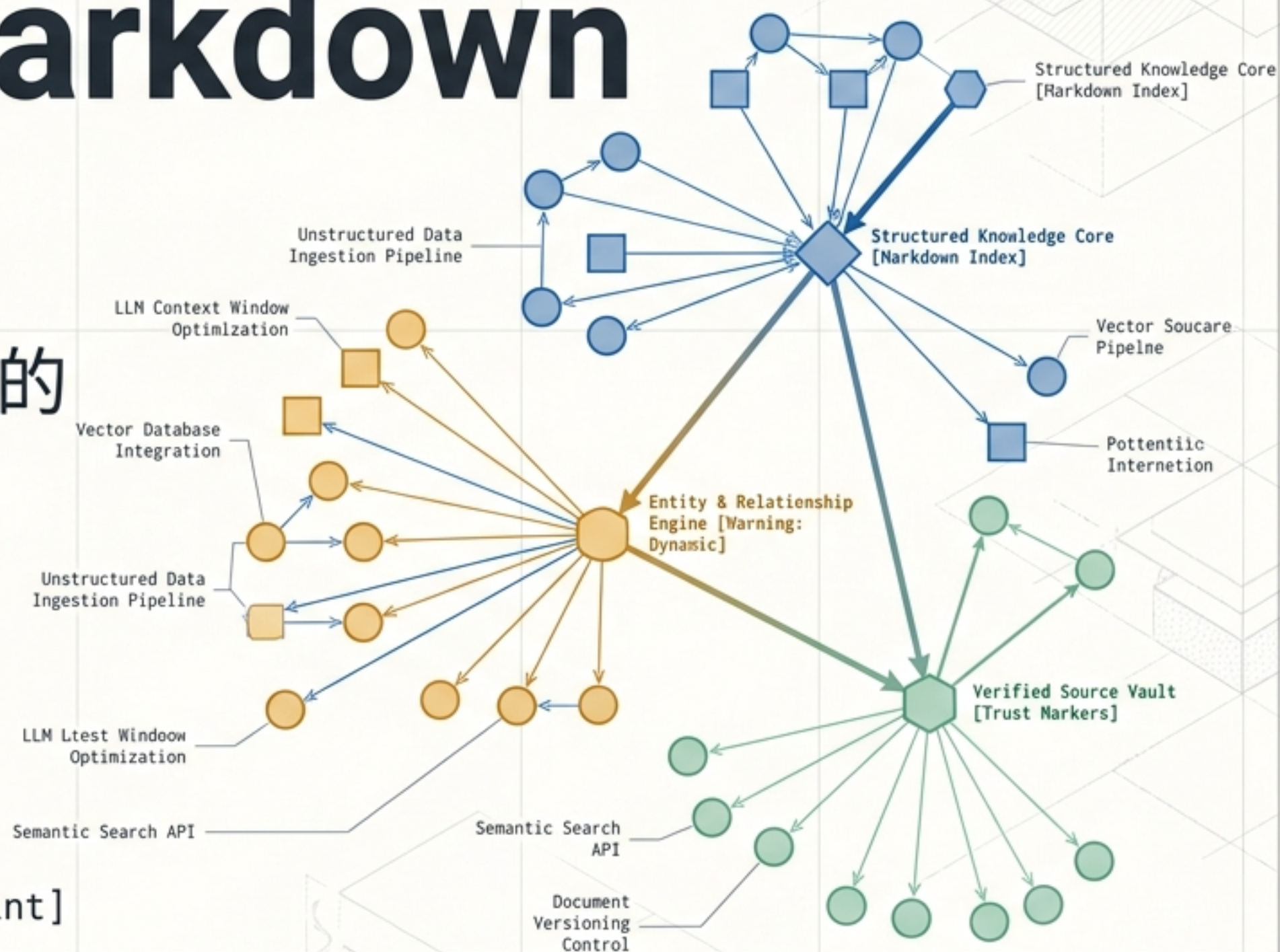


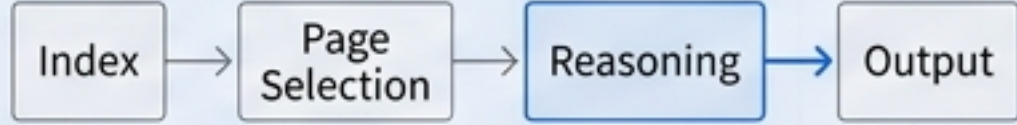
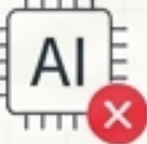
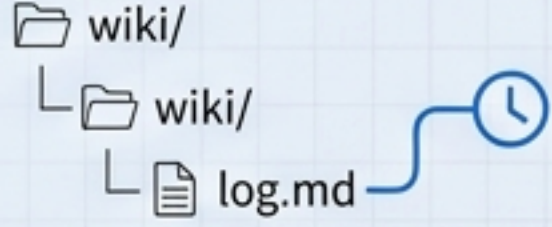
企業級 AI 知識轉型： 索引導向的 Markdown LLM Wiki

從非結構化文件到單一真實來源的
架構藍圖與實踐指南

[Enterprise Architecture & AI Integration Blueprint]



為什麼傳統 RAG 救不了企業文件？

	現狀：傳統文件庫	痛點：通用型 RAG	破局：LLM Wiki (Index-based)
尋找方式	人工拼湊分散的 SA、附件與排程	盲目向量檢索，容易與「單一真實來源」脫鉤	「先選頁、再推理」的索引導向 
AI 推論基礎	AI 無法存取 	無出處推論嚴重	基於持久化結構化知識層 
資訊可信度	依賴個人經驗	無法保證跨文件一致敘述	強制引用 <code>[[sources/...]]</code> 與不確定性標示  <code>[[sources/docA.pdf#p4]]</code>
變更追溯	無跨檔追溯	黑箱機制 	wiki/log.md 完整追溯 

典範轉移：從「盲目檢索」到「索引優先 (Index-First)」

Volume > Structure

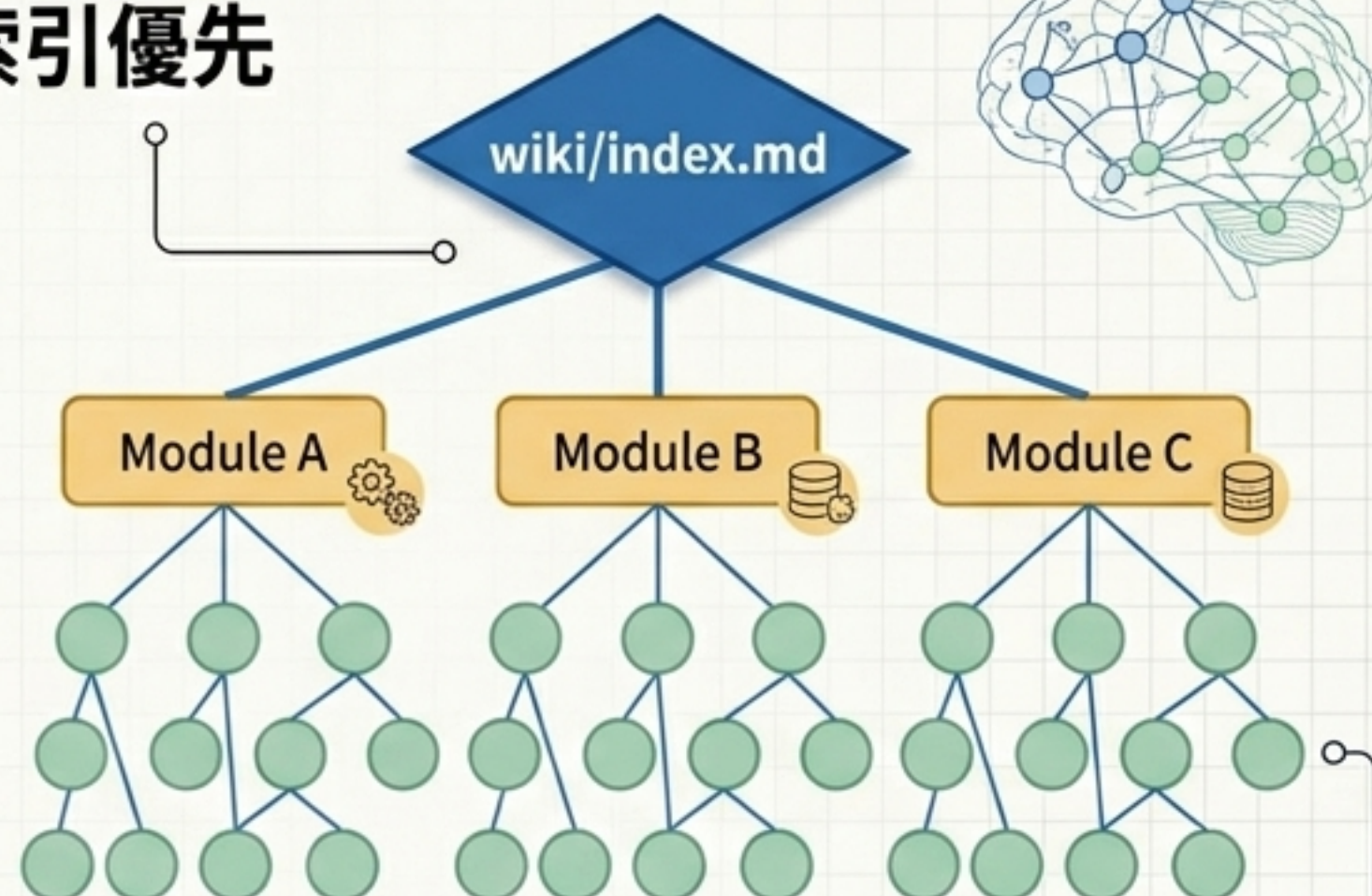
盲目檢索



每次從零翻長文，
依賴黑箱 Embedding。

Navigation > Volume

索引優先



以 `wiki/index.md` 為導覽入口。
Agent 與人類皆需「先選頁，再提取」，
建立如同神經網路般的目錄契約。

持久化知識層的目錄契約 (Directory Contract)

Layer 3
Control Panel

Layer 2
Knowledge Nodes

Layer 1
Base foundation

wiki/lint/
(矛盾診斷)

wiki/index.md
(總目錄)

wiki/log.md
(僅可 Append 的操作日誌)

sources/
(來源摘要)

queries/ & faq/
(可重用問答)

entities/
(具體系統/外部方)

raw/

不可變的原始歸檔 (Immutable).
既有檔案絕對不就地竄改。

知識頁面解剖圖：一張完美 Grounded Grounded 的 Wiki 頁面

YAML Frontmatter

強制屬性標籤 (Metadata)，
包含狀態與更新頻率。

```
---  
title: 'OPEN錢包保費支付流程'  
type: concept  
status: active  
---
```

OPEN錢包保費支付流程

Relationships

* governed_by: [[entities/NCCC]] ←

狀態一致性設計

當回傳逾時，系統應顯示「處理中」而非逕判失敗。(確定) [[sources/open錢包行動支付導入]]

若等待窗口關閉，則觸發人工覆核。(推測)

雙向連結 (Bi-directional Links)

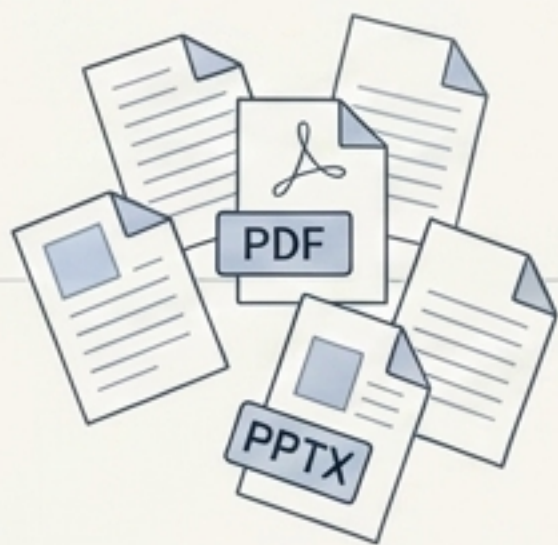
顯式圖邊 [[entities/...]]，
建構 Agent 推理的網狀路徑。

信任框架 (Trust Markers)

句尾強制附加信心水準 (確
定/推測/未知) 與追溯來源，
徹底消滅無出處推論。

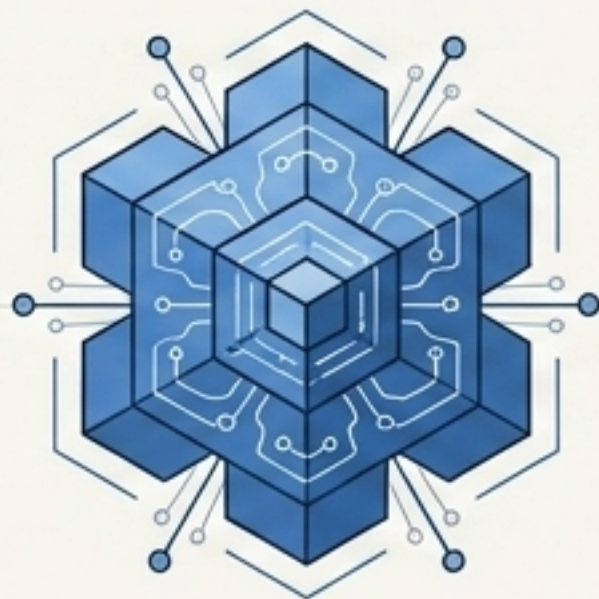
雙引擎 workflow 之首：Ingest 資料精煉管線

1. 來源攝取



Unstructured Input

2. 多模態預處理



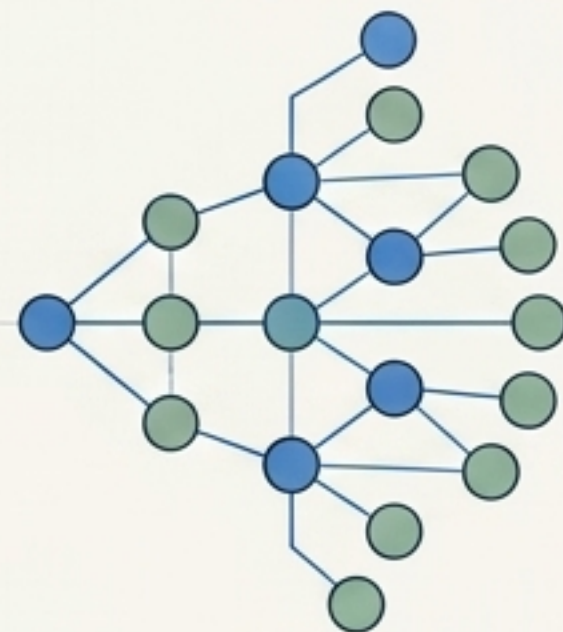
AI Refinement Engine
運用多模態模型解析複雜版面與流程圖，轉化為高正確度的 Markdown。

3. 歸檔納管



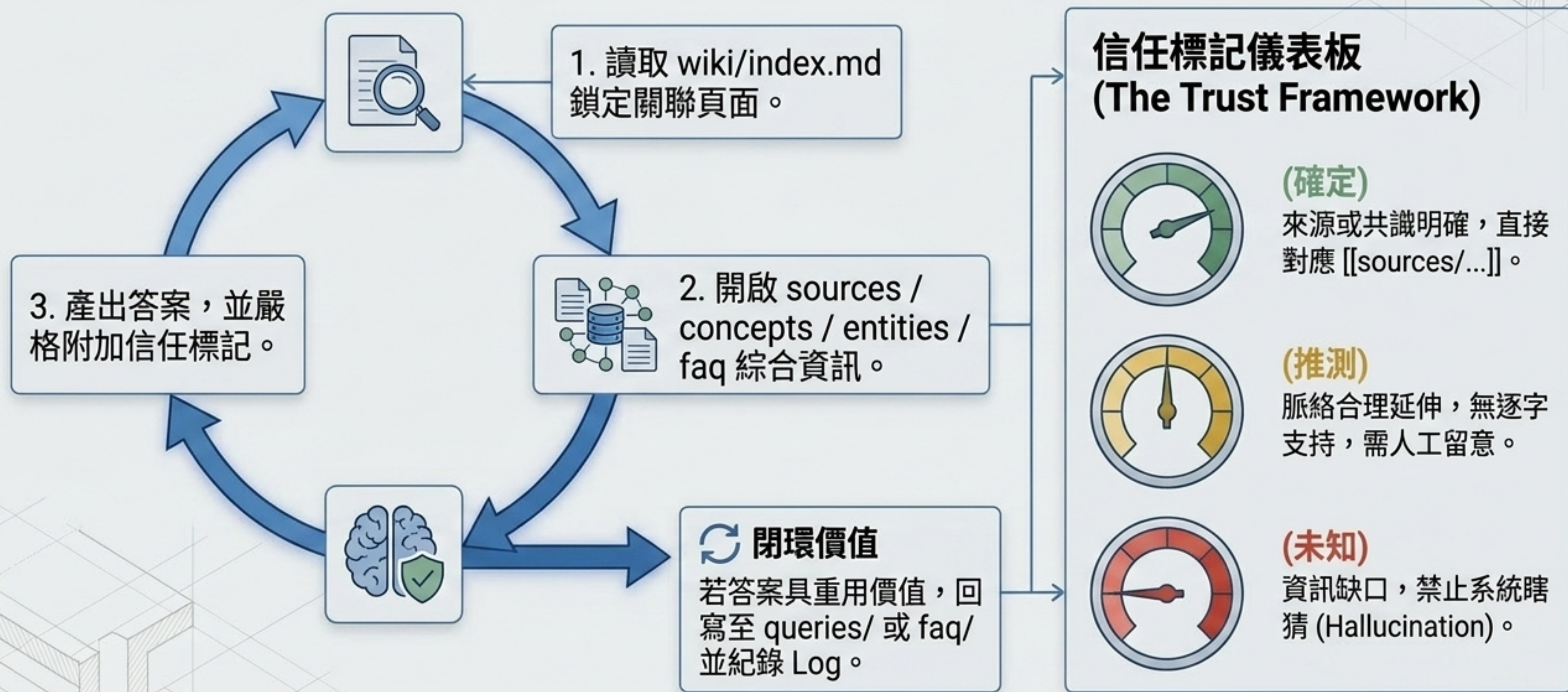
以「新增檔案」寫入
raw/sources/*.md
(不可竄改)。

4. 知識收斂

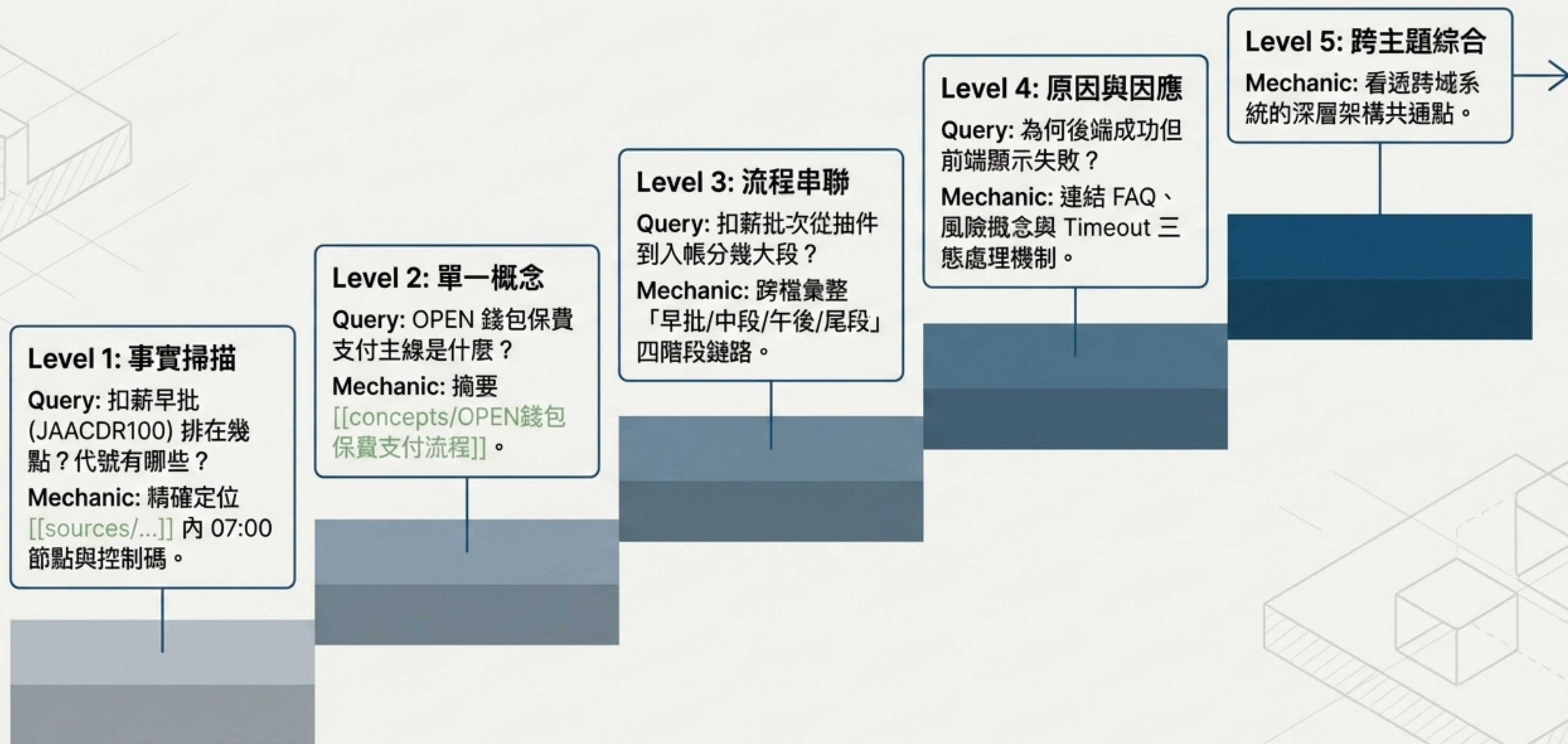


抽離 Concepts & Entities
→ 補齊雙向連結
→ 更新 index.md
→ Append log.md

雙引擎 workflows 之二：Query 提取與信任機制

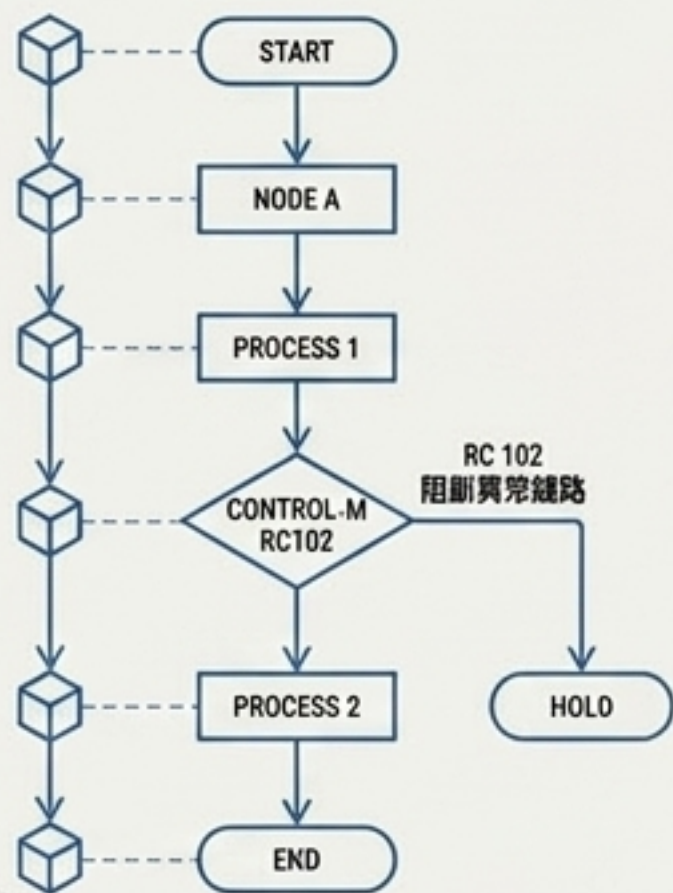


實戰驗證：AI 推理能力階梯 (The Capability Staircase)



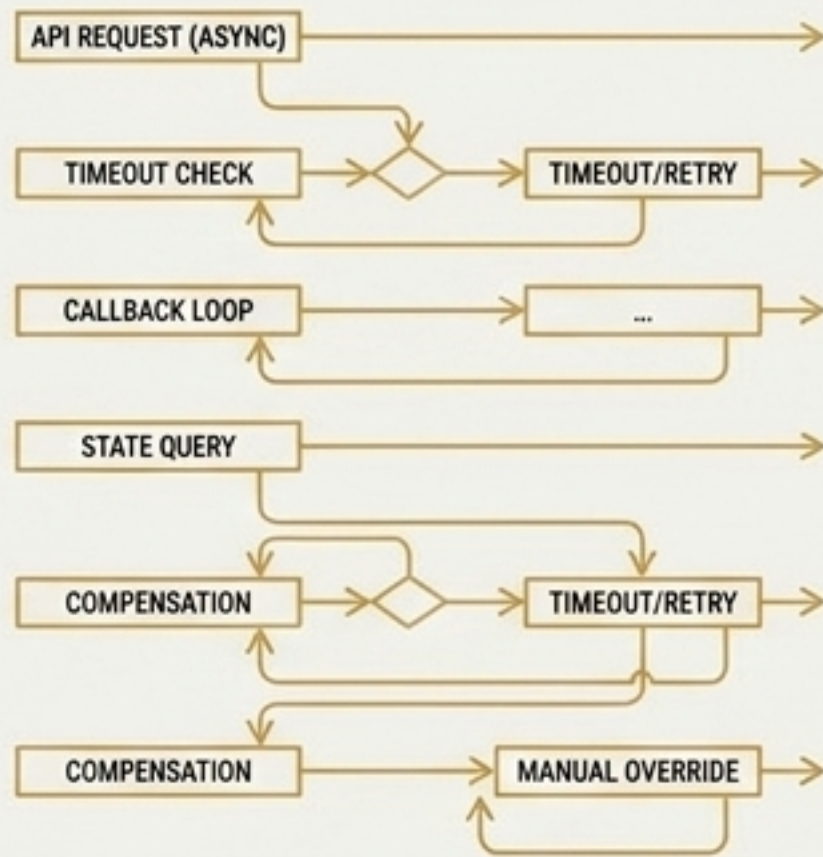
終極洞察合成：跨系統邊界的狀態一致性

扣薪批次排程控制 (Batch Control)



- 機制：固定時序與節點推進。
- 中斷條件：依賴控制碼 (如 RC 102) 阻斷異常鏈路。

OPEN 錢包風險治理 (Payment Risk)



- 機制：處理跨系統即時回傳與 Timeout。
- 中斷條件：依賴狀態頁面 (成功/失敗/處理中) 與人工覆核補償。

統一架構邏輯 (The Shared Core)

無論是「批次排程」還是「即時支付」，核心治理皆是為了在跨系統邊界中確保「與」異常補償機制」。LLM Wiki 能看透表層規格，找出底層架構的共通軸。

企業級 AI 大腦的投資報酬 (ROI) 與最終願景



LLM Wiki = 目錄化結構 + 強制連結引用 + 演進日誌 (Append-only)